EXPO MOBILE / IMPLEMENTATION REPORT

Mental Notes Mobile App

What is implemented, how the mobile client works, and what remains before a dependable release build. This report covers the Expo mobile app only.

APP STATUS	BUILD SETUP	CURRENT CHECKS
Working mobile MVP SDK 57 / React Native / TypeScript	**Expo project linked** Android APK preview build submitted	**Passed** TypeScript and Android bundle export
TESTING	**API CONFIG**	**NEXT**
Phone sign-in and assessment flow confirmed in Expo Go.	APK points at temporary API tunnel.	Deploy mobile auth API before release.

1. What was changed

A separate **expo-mobile/** application was created alongside the existing web app. It uses Expo Router for screen navigation and React Native components and styles. Mobile screens were implemented for sign-in, account creation, dashboard, assessments, results, history, profile, and settings. The mobile app is aligned to **Expo SDK 57** for the Expo Go version used during phone testing.

The mobile dashboard includes a native progress chart for PHQ-9 and GAD-7 results, with questionnaire selection, comparison view, score scale, dates, and empty/single-result states. It reads the user's assessment history.

2. What was implemented

- **Navigation:** file-based Expo Router routes, protected-screen redirect, and mobile-friendly links and buttons.
- **Authentication:** login, registration, password-reset request, verification resend, sign-out, and protected sessions using a seven-day bearer JWT stored with Expo SecureStore.
- **Assessments:** PHQ-9 and GAD-7 answer flows and API submission; score calculation and persistence remain on the existing backend.
- **Results and history:** detailed result view, recent check-ins, history list, and form filters.
- **Account features:** profile view/edit and settings actions, including result deletion and confirmed account deletion.
- **API integration:** Axios bearer-token injection and existing assessment API contracts.
- **Native UI:** View, Text, Pressable, TextInput, ScrollView, FlatList, StyleSheet, and SVG rendering replace web DOM, CSS, React Router, and Recharts.

Reusable business logic includes existing API contracts, server-side scoring and persistence, assessment definitions and severity interpretation, and product flow. Browser-only components and styling were rewritten for native navigation and presentation.

APP FLOW / DATA AND AUTH

Screens and architecture

SCREEN	MOBILE IMPLEMENTATION
Login / signup	Native fields, validation, bearer-session storage, consent, and password-reset entry.
Dashboard	Latest scores, change text, verification notice, progress chart, and recent check-ins.
Assessment	Questionnaire answer flow and API submission; server calculates the score.
Results / history	Saved result details and previous check-ins with type filters.
Profile / settings	Profile details and account/result management with confirmation for destructive actions.

Request and session flow

The app calls the existing Express API. Native login and registration return a signed bearer token stored in platform secure storage; subsequent calls send it in the Authorization header. The web client retains its HTTP-only cookie flow. Assessment scores are calculated and stored on the backend, while the mobile app renders results and history.

Development connectivity

Phone testing used Expo Go through an Expo tunnel and a temporary API tunnel because Windows classified the Wi-Fi network as Public. The mobile app reads EXPO_PUBLIC_API_URL. EAS cloud builds use the preview environment variable, not the ignored local .env file.

RELEASE BLOCKER

The stable hosted API health endpoint responds, but POST /api/auth/mobile/session currently returns 404 there. Mobile bearer-auth routes work in the local backend used for phone testing. Deploy the mobile auth changes before an APK can authenticate reliably against the stable backend.

Validation completed

- **TypeScript:** npm run typecheck passed.
- **Expo dependencies:** npx expo install --check passed.
- **Android bundle:** npx expo export --platform android passed.
- **Phone flow:** sign-in and assessment completion confirmed in Expo Go; dashboard chart was added and type/bundle checks passed.

NEXT / RELEASE READINESS

What still needs to be added

The core app flow is implemented. These items close the gap between the tested Expo Go MVP and a dependable standalone release.

PRIORITY	REMAINING WORK	WHY IT MATTERS
1 - Required	Deploy mobile bearer-auth routes and middleware to the stable backend; verify login, registration, protected calls, and password-reset invalidation.	The current hosted API does not recognize the mobile session endpoint.
1 - Required	Replace the temporary API tunnel with a durable URL in EAS preview/production and rebuild.	The tunnel is temporary and stops when its local service closes.
2 - Release	Add final app icons/splash, privacy/support URLs, version/build-number policy, and production build profiles.	Needed for a branded and maintainable release.
2 - Release	Review bearer-token lifecycle, privacy disclosures, data deletion, accessibility, and platform behavior.	The seven-day stateless token is cleared on local sign-out but has no server revocation endpoint.
3 - Quality	Check slow/offline networks, expired sessions, small/large screens, empty/long histories, and screen-reader navigation on devices.	Bundle checks do not replace interactive device QA.
3 - Distribution	Prepare store-ready Android AAB and iOS archive/TestFlight workflow after production API and signing setup.	Internal APK testing and store distribution use different signing paths.

Build options

Android: The Expo project is linked and an internal preview profile is configured to build an APK. EAS generated Android signing credentials and the preview build has been submitted. This APK embeds a temporary API tunnel URL and depends on the local server/tunnel. Use it for short-term app testing only; rebuild after mobile auth is deployed to the stable backend.

iOS: EAS can build remotely from Windows. An installable iPhone IPA requires Apple signing and ad hoc provisioning for the device, or a TestFlight/App Store path. A simulator build is separate and requires a Mac simulator to run.

Recommended completion order

Deploy the mobile auth routes, verify them on the hosted API, replace the temporary preview API URL with the stable endpoint, and rebuild the APK. Then set up iOS signing and device distribution. Prepare store builds after the production API and release metadata are ready.

Scope note: this report covers the Expo mobile application and its direct API dependency. It does not report on the web app.

BUILD STATUS UPDATE

Android APK build

CURRENT RESULT	No APK artifact yet
BUILD ATTEMPTS	The initial build failed because Gradle could not deserialize Expo autolinking metadata: **expo-font** resolved to SDK 58. The next attempt used an isolated dependency tree, but inherited the same mismatch.
FIX APPLIED	Pinned Expo Font to the SDK 57 compatible release and added the SDK 57 compatible React Native Worklets peer dependency. **Expo Doctor now passes 21/21 checks.**
CURRENT BUILD	A fresh Android preview APK build was submitted to EAS after the dependency fix. It is queued or compiling; its outcome and download artifact are not available yet.

Build links and next steps

Android EAS build (latest): Open build page

Android build log (first attempt): Open build page

What remains before phone testing

Wait for the queued build to finish and download the APK from the latest EAS build page. The current preview API URL is a temporary tunnel to the local backend; keep that server and tunnel online for sign-in. For dependable testing and release, deploy the mobile bearer-auth routes and point EAS at the stable API first.

iOS: internal IPA distribution still needs Apple signing credentials and a registered device, or a TestFlight/App Store setup.